



PTCAS: Prompt tuning with continuous answer search for relation extraction

Yang Chen^{a,*}, Bowen Shi^b, Ke Xu^a

^a State Key Lab of Software Development Environment, Beihang University, Beijing, 100191, Beijing, China

^b School of Journalism, Communication University of China, Beijing, 100024, Beijing, China

ARTICLE INFO

Keywords:

Prompt tuning

Relation extraction

Few-shot learning

Pretrained language model

ABSTRACT

Tremendous progress has been made in the development of fine-tuned pretrained language models (PLMs) that achieve outstanding results on almost all natural language processing (NLP) tasks. Further stimulation of rich knowledge distribution within PLMs can be achieved through additional prompts for fine-tuning, namely, prompt tuning. Generally, prompt engineering involves prompt template engineering, which is the process of searching for an appropriate template for a specific task, and answer engineering, whose objective is to seek an answer space and map it to the original task label set. Existing prompt-based methods are primarily designed manually and search for appropriate verbalization in a discrete answer space, which is insufficient and always results in suboptimal performance for complex NLP tasks such as relation extraction (RE). Therefore, we propose a novel prompt-tuning method with a continuous answer search for RE, which enables the model to find optimal answer word representations in a continuous space through gradient descent and thus fully exploit the relation semantics. To further exploit entity-type information and integrate structured knowledge into our approach, we designed and added an additional TransH-based structured knowledge constraint to the optimization procedure. We conducted comprehensive experiments on four RE benchmarks to evaluate the effectiveness of the proposed approach. The experimental results show that our approach achieves competitive or superior performance without manual answer engineering compared to existing baselines under both fully supervised and low-resource scenarios.

1. Introduction

Relation extraction (RE), one of the most fundamental tasks in natural language processing (NLP), aims to detect the relationships between the given entities contained within a sentence. Many downstream applications can benefit from the structured knowledge captured by RE, including the completion of knowledge graphs [1], dialogue systems [2], and question-answering [3]. As pretrained language models (PLMs) [4,5] have been developed, their fine-tuning has become a dominant approach to RE in recent years. However, pretrained knowledge may not be effective for downstream tasks because of the objective gap between pretraining and fine-tuning. To address this issue, prompt-tuning approaches [6–11] that use additional prompts to fine-tune PLMs have recently been proposed and have been demonstrated to be effective, particularly in situations where resources are limited. By restructuring

* Corresponding author.

E-mail address: yangchen11235@gmail.com (Y. Chen).

input examples into cloze-style phrases and mapping labels to candidate answer words, downstream tasks can be aligned more closely with the pre-training tasks.

Generally, the full procedure of prompt engineering consists of prompt template engineering, which is the process of searching for a perfect template for a specific task, and answer engineering, the goal of which is to find an answer space and map it to the original task label set. For certain NLP tasks whose task label space is not complex, such as text classification, mapping a simple answer space to the task label space is effective. However, for certain tasks, such as RE, which have a complex semantic label space, a straightforward answer space is insufficient and always results in suboptimal performance. Defining a complex answer space and mapping it to the original task label set for RE remains challenging. Recent prompt-based methods [12,13] for RE mostly manually design and search for appropriate verbalizations in a discrete answer space. These methods tend to be labor-intensive because of the need for a handcrafted answer-word space for relation labels. More importantly, searching empirically for a discrete answer space may make it difficult to arrive at the optimal point. Moreover, because of the importance of entity-type information in RE, integrating structural relation semantics with typical prompt-tuning methods is worthy of further investigation.

In this paper, we propose a prompt-based method with a continuous answer search for relation extraction that enables the model to find optimal answer word representations in a continuous space through gradient descent, thoroughly exploiting relation semantics. Specifically, we first decompose a canonical relation into three components: head entity, relation, and tail entity types. Then, we transform each input sentence into a cloze-style phrase, where we set [MASK] tokens in front of both the head and tail entity mentions, as well as between entity mentions that are expected to encode the relation semantics. Additionally, we embed each original relation label and entity type into its own continuous vector space and then use their embeddings to calculate scores with the outputs of the used PLM in the [MASK] position. The scores of the entity types and relation labels are jointly used to predict the final relation distribution. To further exploit entity-type information and integrate structured knowledge into our approach, we added an additional TransH-based structured knowledge constraint to the optimization procedure, which significantly improved the model's performance. To evaluate the effectiveness of our approach, we conducted comprehensive experiments under both fully supervised and low-resource scenarios using four widely used RE benchmarks. Compared to the fine-tuning and prompt-tuning methods, especially the prompt-tuning models searching for answer words in a discrete space, our method achieves competitive or superior performance in both fully supervised and low-resource scenarios without manual answer engineering, which proves our model's ability to automatically search for optimal answer word embedding in a continuous space. The contributions of this study can be summarized as follows:

- We propose a novel prompt tuning method with continuous answer searching for relation extraction (RE) that enables the model to determine the optimal answer word embeddings in a continuous space through gradient descent; thus, it fully exploits relational semantics. Our approach presents a novel learning paradigm applicable to real scenarios.
- To further exploit entity-type information and integrate structured knowledge into our approach, we design and add a TransH-based structured knowledge constraint to the optimization procedure, significantly improving the performance of the model.
- We conduct comprehensive experiments on four RE benchmarks to evaluate the effectiveness of our approach. The experimental results show that our approach achieves competitive or superior performance without manual answer engineering compared to existing baselines under both fully supervised and low-resource scenarios.

2. Related work

In this section, we discuss related works from two perspectives: RE and prompt tuning, both of which are involved in the proposed method.

2.1. Relation extraction (RE)

RE focuses on identifying the semantic relationship between two defined entities according to their context in a sentence and plays a crucial role in information extraction and knowledge-based construction. Early RE attempts included pattern-based [14], CNN/RNN-based [15] and graph-based [16] models. Since the development of pretrained language models in recent years, PLMs have become a standard component of RE systems, and the fine-tuning of PLMs for RE has achieved promising results. Moreover, various knowledge-enhanced PLMs have been proposed to incorporate external knowledge into RE tasks, such as KNOWBERT [17], SPANBERT [18], LUKE [19], and MTB [20].

In addition to the above research, numerous studies [21–23] have focused on RE in few-shot settings, where limited or few annotated instances of the training dataset are available. Generally, few-shot RE can be categorized as metric- or optimization-based. Metric-based approaches measure the similarity distances of embeddings in latent spaces to classify semantic relations, drawing considerable attention in academia because of their encouraging performance. Snell et al. [24] first proposed a prototype network, which is an effective classification model based on the notion that each class has a prototype determined by averaging the embedding vectors of all the instances of the support set. Yang et al. [25] utilized the interplay between local- and instance-level information to generate a richer semantic representation. Liu et al. [26] adopted a direct-join mechanism for relational information. Liu et al. [27] proposed an effective and parameter-free prototype rectification method that rectifies the original prototypes based on their relational information.

2.2. Prompt tuning

Notwithstanding the popularity of fine-tuning PLMs, a wide gap remains between the objectives of pretraining and fine-tuning. The development of prompt-tuning methods was triggered by the advent of GPT-3 [7] and has led to outstanding performance in a wide range of NLP tasks. Typical prompt-tuning methods transform downstream tasks into masked language model tasks by inserting cloze-style templates and mapping task labels to the corresponding answer words. Following Liu et al. [6], prompt engineering comprises prompt template engineering, which is the procedure for searching for a perfect template for a specific task, and answer engineering, which seeks an answer space and maps it to the original task label set. Manually creating intuitive templates based on human insights is the most natural method for creating prompts. Petroni et al. [28] explored knowledge of PLMs by Handcrafted cloze-style templates. Brown et al. [7] developed handcrafted prefix prompts for various tasks. Schick and Schütze [29,8] utilized predefined templates in a few-shot learning scenario for text classification and conditional text generation. Automated searches for discrete prompts have been extensively investigated to reduce the requirement for handcrafted prompts. Gao et al. [30] applied the seq2seq pretrained language model T5 [31] for the automatic generation of templates. Shin et al. [32] used downstream application training samples to identify template tokens by using a gradient-guided search. Recent research [10,33,9] proposed soft prompt methods that search for learnable continuous embeddings as prompt tokens. Most answer engineering studies have focused on discrete answer searches. Cui et al. [34] and Yin et al. [35] manually designed answer words and mapped them onto a constrained task label set. Jiang et al. [36] expanded the initial answer space by using a paraphrasing strategy. Few studies have explored the feasibility of optimizing soft-answer tokens using gradient descent. Hambardzumyan et al. [37] and Chen et al. [38] defined a virtual token corresponding to each task label, which was subsequently optimized to avoid manual answer engineering.

Regarding prompt-tuning methods related to relation extraction, PTR [12] was proposed as a prompt-tuning method for RE by combining subprompts into task-specific prompts based on logic rules. KnowPrompt [38] was proposed to integrate the implicit knowledge of relation labels into prompts by constructing prompts with learnable virtual-type words. FPC [13] was proposed for relation prompt learning with a prompt curriculum that adds a cloze-style auxiliary task, enabling the model to capture the semantics of relation labels. GenPT [39] was proposed as a generative prompt tuning method that transforms relation extraction into an infilling problem.

3. Preliminaries

We begin with some prerequisites before introducing the PTCAS. First, we provide a formal definition of sentence-level RE, as follows: Formally, a sentence-level RE task can be formulated as $D = \{\mathcal{X}, \mathcal{Y}\}$, where $\mathcal{X}$ is the set of sentence examples, and $\mathcal{Y}$ is the corresponding set of relation labels. For every example $x \in \mathcal{X}$ that comprises a token sequence $\{w_1, w_2, w_3, \dots, w_n\}$ and the spans of two given entities, the task aims to predict the relation label $y \in \mathcal{Y}$ between the entities.

3.1. Fine-tuning for RE

According to the PLM, fine-tuning methods usually first convert the token sequence $\{w_1, w_2, w_3, \dots, w_n\}$ into an input sequence for PLM such as $\{[CLS], w_1, w_2, w_3, \dots, w_n, [SEP]\}$. Additionally, entity markers were inserted into the token sequence to index the positions of the entities. Specifically, special tokens are inserted at the start and end of the entity span, such as “[E]” and “[/E]”. It is possible to use type markers by integrating entity type information into the markers if the annotation of the entity type is available. Subsequently, the PLM encodes all the tokens of the input sequence into the corresponding vectors $\{h_{[CLS]}, h_1, h_2, h_3, \dots, h_n, h_{[SEP]}\}$. A classifier outputs the probability distribution over the label set $\mathcal{Y}$ based on the concatenation of the vectors of the two start markers. The parameters of the PLM and classifier were fine-tuned by minimizing the cross-entropy loss for the entire dataset.

3.2. Prompt tuning of PLMs

To alleviate the discrepancy between pre-training and downstream tasks, diverse prompt-tuning methods have been proposed. Typically, a prompt is composed of a template $\mathcal{T}(\cdot)$ and verbalizer. By designing and adding additional tokens and [MASK] tokens, template $\mathcal{T}(\cdot)$ transforms the original input x into a cloze-style phrase $\mathcal{T}(x)$. The verbalizer was expressed as a map $\phi: \mathcal{Y} \rightarrow \mathcal{V}$, where $\mathcal{Y}$ refers to the label set and $\mathcal{V}$ refers to the answer word space in the vocabulary of the used PLM. Consequently, the classification task is converted into a mask language model task with the objective of predicting the probability distribution of the [MASK] tokens.

4. Methodology

In this section, we introduce PTCAS, a novel prompt-based method with continuous answer search for RE. Fig. 1 presents an overview of the proposed method.

4.1. Relation decomposition and prompt template construction

Generally, the two entity types of a relationship can be inferred from their basic meanings. For instance, given the relation “*org:founded by*”, it is obvious that the head and tail entity type matching the relation belong to “*organization*” and “*person*”, respectively. Meanwhile, knowledge of a relation’s entity types helps identify and separate perplexing relations that have similar semantic

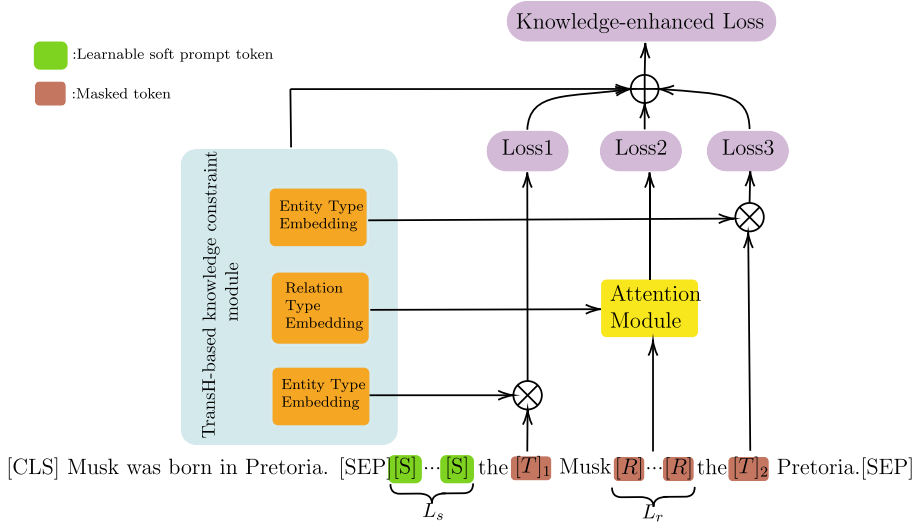


Fig. 1. Model architecture of our proposed PTCAS.

contexts. Inspired by the above observations, the canonical relation R between two entities can be formulated as $R = (h, r, t)$, where h and t are the head and tail entity types, respectively, and r is the relationship type. We defined a relation as correct only when its head entity, tail entity, and relation type were correct. Based on the above relation decomposition, we constructed our prompt template with the knowledge of entity types. As shown in Fig. 1, given a sentence $s = \{w_1, w_2, w_3, \dots, w_n\}$, we used $[E]_1$ and $[E]_2$ to denote mentions of the head and tail entities, respectively. Hence, the template function $\mathcal{T}$ transforms the original input sentence into $\mathcal{T}(s)$, which can be formulated as follows:

$$\mathcal{T}(s) = \{[CLS]s[SEP]\underbrace{[S] \dots [S]}_{L_s}the[T]_1\underbrace{[E]_1 \dots [E]_2}_{L_r}the[T]_2[SEP]\}, \quad (1)$$

where $[T]_1$ and $[T]_2$ are [MASK] tokens for predicting the answer words of the head and tail entity types, respectively; $\underbrace{[R] \dots [R]}_{L_r}$ represents the n_r [MASK] tokens for predicting answer words of relation types between the two marked entities; and $\underbrace{[S] \dots [S]}_{L_s}$ represents the n_s soft prompt tokens, which are learnable virtual prompt words during training.

4.2. Continuous answer search

The prompt tuning process in previous studies has typically been based on the manual or automatic generation of a one-to-one mapping between a discrete answer space and a relation label set, which maintains the high computational complexity of the search process while not utilizing the abundant semantic information in the relation labels. To overcome these limitations, we propose a continuous answer search mechanism that constructs a mapping from a continuous answer space to a relation-type set. Specifically, by using the relation decomposition introduced in the previous section, we can obtain the entity- and relation-type sets from the original relation set. Subsequently, the entity- and relation-type sets are embedded into their respective consecutive vector spaces. The embedding space of entity types, denoted as P , is an $n_t \times d$ matrix, where n_t is the number of entity types and d is the dimension of each entity type vector. Similarly, the embedding space of the relation labels, denoted as Q , is an $n_r \times d$ matrix, where n_r is the number of relation labels and d is the dimension of each relation type. As shown in Fig. 1, the input sentence with the constructed template is first fed into a PLM $\mathcal{L}$, such as *RoBERTa_{LARGE}* [5]. Based on the output hidden vectors of $\mathcal{L}$, we define $p(n | P; \mathcal{L})$ as the probability of the n th entity type for a [T] masked token, which is calculated as follows:

$$p_e(n | P; \mathcal{L}) = \frac{\exp(h_{[T]} \cdot P_n)}{\sum_{i=0}^{n_t-1} \exp(h_{[T]} \cdot P_i)} \quad (2)$$

where $h_{[T]}$ is the hidden output vector of the [T] token in the entity-type position.

Attention on [R] Tokens: The output hidden embedding of each [R] token in the template is the corresponding vector representation of each answer word. In our setting, there were multiple answer words for each relationship type. Intuitively, for a specific relationship type, the answer words have varying importance and should be assigned different weights. Therefore, we applied an attention mechanism to predict the relationship types. The probability of the n th relation type is computed as

$$p_r(n | Q; \mathcal{L}) = \frac{\exp(\hat{h}_r^n \cdot Q_n)}{\sum_{i=0}^{n_r-1} \exp(\hat{h}_r^i \cdot Q_i)} \quad (3)$$

$$\hat{h}_r^i = \sum_{j=0}^{L_r-1} a_j^i \cdot h_r^j \quad (4)$$

$$a_j^i = \frac{\exp(Q_i \cdot h_r^j)}{\sum_{j=0}^{L_r-1} (Q_i \cdot h_r^j)}, \quad (5)$$

where h_r^j is the hidden output vector of the j th [R] token in the template. Finally, the cross-entropy loss for the entire dataset is formulated as follows:

$$\mathcal{L}_{MLM} = -\frac{1}{|\mathcal{X}|} \sum_{x \in \mathcal{X}} \log(p_e(y_h | x)) + \log(p_r(y_r | x)) + \log(p_e(y_t | x)), \quad (6)$$

where the relation $y = (y_h, y_r, y_e)$ is the label decomposition of an instance, y_h is the y_h -th entity type for the head entity, y_t is the y_t -th entity type for the tail entity, and y_r is the y_r -th relation type.

4.3. Joint optimization with TransH-based constraint

To integrate structural knowledge into our approach, we added knowledge-structured constraints to the loss function optimization. One straightforward approach is to use knowledge constraints from a graph embedding method such as TransE [1], which has been adopted in recent studies [38]. However, considering that different relation types may have the same head and tail entity types in practical settings, we adopted our TransH-based constraint inspired by TransH [40], which supports the distributed representations of an entity across multiple relations. Rather than placing the relation-specific translation vector $\mathbf{r}$ in the same space as the entity embeddings, TransH places it in the relation-specific hyperplane $\mathbf{w}_r$ for the relation type $\mathbf{r}$, where $\mathbf{w}_r$ is normalized. In our task, for the relation $\mathbf{R}$, we can obtain its embedding: $\text{Embed}(\mathbf{R}) = (\mathbf{h}, \mathbf{r}, \mathbf{t})$, where $\mathbf{h}$ is the embedding of the head entity, $\mathbf{r}$ is the embedding of the relation, and $\mathbf{t}$ is the embedding of the tail entity. The entity-type embeddings $\mathbf{h}$ and $\mathbf{t}$ are first projected onto hyperplane $\mathbf{w}_r$, and their derived projections are denoted by $\mathbf{h}_\perp$ and $\mathbf{t}_\perp$, respectively. If $(\mathbf{h}, \mathbf{r}, \mathbf{t})$ is a ground-truth relation, then the score function $f_r(h, t) = \|\mathbf{h}_\perp + \mathbf{r} - \mathbf{t}_\perp\|_2^2$ is expected to be sufficiently small. Formally, we can reformulate the above procedure as follows:

$$\mathbf{h}_\perp = \mathbf{h} - \mathbf{w}_r^\top \mathbf{h} \mathbf{w}_r \quad (7)$$

$$\mathbf{t}_\perp = \mathbf{t} - \mathbf{w}_r^\top \mathbf{t} \mathbf{w}_r \quad (8)$$

$$f_r(\mathbf{h}, \mathbf{t}) = \left\| (\mathbf{h} - \mathbf{w}_r^\top \mathbf{h} \mathbf{w}_r) + \mathbf{r} - (\mathbf{t} - \mathbf{w}_r^\top \mathbf{t} \mathbf{w}_r) \right\|_2^2. \quad (9)$$

To encourage discrimination between the golden and incorrect triplets of relation decomposition, we define the loss $\mathcal{L}_{know}$ of the implicit knowledge-structured constraint as follows:

$$\mathcal{L}_{know} = -\log \sigma(\gamma - f_r(h, t)) - \sum_{i=1}^n \frac{1}{n} \log \sigma(f_r(h'_i, t'_i) - \gamma), \quad (10)$$

where (h'_i, r, t'_i) are the negative samples, σ denotes the sigmoid function, γ denotes the margin, and $f_r(\mathbf{h}, \mathbf{t})$ is the score function. To construct negative samples, we randomly sampled n pairs of incorrect entity types (h', t') for relation type r . In the optimization procedure, the total loss was computed as follows:

$$\mathcal{L} = \mathcal{L}_{MLM} + \lambda \mathcal{L}_{know}, \quad (11)$$

where λ is the hyperparameter.

4.4. Prototype initialization

For some complex relations in an RE dataset, humans can easily recognize their essential meaning because it can be expressed by one word, namely, a prototype word. For example, the prototype words “*per:country_of_birth*” and “*per:city_of_birth*” are both “*born*”, which is the common meaning for the two relations. We also used prototype words to describe entity types. More examples of prototype words are given in Table 1. These prototype words are easy to design and are sufficient to capture basic semantic information in complex relationships. Hence, we used the embedding of the prototype word in the language modeling head of the PLM to initialize the corresponding relation embedding. In this manner, relation embeddings start from an explicit semantic point and are then optimized to arrive at an optimal point.

4.5. Training and inference

During training, we used the prototype words of relation decomposition to initialize the entity- and relationship-type embedding layers. Moreover, we adopted a two-stage optimization strategy. First, we optimized the parameters of the TransH-based constraint

Table 1

Prototype words of head entity type, relation type and tail entity type for some relations in the datasets TACRED, TACREV, and ReTACRED.

Relation	Prototype words		
	Head entity type	Relation type	Tail entity type
per:country_of_birth	person	born	country
per:stateorprovince_of_birth	person	born	state
per:city_of_birth	person	born	city
org:country_of_headquarters	organization	located	country
org:stateorprovince_of_headquarters	organization	located	state
org:city_of_headquarters	organization	located	city
org:members	organization	member	organization
no_relation	entity	irrelevant	entity

Table 2

Statistics of the four datasets used in our experiments.

Dataset	#Train	#Dev	#Test	#Rel
TACRED	68,124	22,631	15,509	42
TACREV	68,124	22,631	15,509	42
Re-TACRED	58,465	19,584	13,418	40
SemEval	6,507	1,493	2,717	19

with a high learning rate lr_1 , whereas the other parameters were frozen. Second, the parameters of the pretrained model language, type of embedding layer, and soft prompt tokens are optimized with a small learning rate lr_2 , medium rate lr_3 , and large rate lr_4 , while the other parameters are frozen.

During inference, the final classification of relation R depends on the sum of the head entity, tail entity, and relation-type scores, which are formulated as follows:

$$score(R) = p_r(k | Q; \mathcal{L}) + \beta[p_e(i | Q; \mathcal{L}) + p_t(j | Q; \mathcal{L})] \quad (12)$$

where k is the index of R in the relation set; i and j are the indices of the head and tail entity types in the entity type set, respectively; and β is a hyperparameter that controls the weight of the knowledge of the entity type over the entire prediction. Finally, the relationship with the highest score is predicted.

5. Experiments

This section presents the details of our experiments, including the datasets, settings, baselines, hyperparameters, and experimental results.

5.1. Datasets and experimental settings

Following previous studies [12,38,13], we conducted experiments using four widely used benchmarks: TACRED [41], TACREV [42], ReTACRED [43], and SemEval 2010 Task 8 (SemEval) [44]. A more detailed description of the datasets is presented in Table 2. We used micro F1 scores as the standard metric to evaluate our model under both fully supervised and low-resource scenarios. In the fully supervised setting, a full dataset was available for both training and validation. In the low-resource setting, we adopted a random sampling of K cases per relation for training and validation, where K was 8, 16, or 32.

5.2. Baseline models

We compared our model with recent competitive RE methods that can be categorized as fine-tuning or prompt-tuning.

Fine-tuning methods: For the vanilla fine-tuning method, we selected *ROBERTA_{LARGE}* as baseline. Additionally, we compared the proposed method with GDPNet [45], TYP Marker [46], and NLI [47]. In light of the importance of entity information in grasping relational semantics, knowledge-enhanced PLMs, which adopt knowledge bases as additional information to strengthen PLM, have been developed and have received more attention in recent years. Hence, we chose KNOWBERT [17], SPANBERT [18], and LUKE [19] as our baselines for the knowledge-enhanced PLMs. The above methods are typical models that incorporate external knowledge to enhance the input features, learning objectives, and model architectures.

Prompt tuning methods: For prompt tuning methods, we selected PTR [12], KnowPrompt [38], FPC [13], and GenPT [39] as baselines, which can be divided into methods with discrete or continuous answer searching.

Table 3

Experimental results of F1 scores (%) on the different test sets of the RE benchmarks. Note that “w/o” indicates that no additional data were needed for pre-training and fine-tuning, while “w/” means the model used extra data or knowledge bases for tasks. The best results are highlighted.

Model	Answer Space	Extra Data	TACRED	TACREV	ReTACRED	SEMEVAL
Fine-tuning pre-trained models						
<i>ROBERTA_{LARGE}</i>	-	w/o	68.7	76.0	84.9	87.6
GDPNet [45]	-	w/o	70.5	80.2	-	-
TYP Marker [46]	-	w/o	74.6	83.2	91.1	-
NLI [47]	-	w/o	71.0	-	-	-
KNOWBERT [17]	-	w/	71.5	79.3	89.1	89.1
SpanBERT [18]	-	w/	70.8	78.0	85.3	-
LUKE [19]	-	w/	72.7	80.6	90.3	-
Prompt-tuning pre-trained models						
PTR [12]	discrete	w/o	72.4	81.4	90.9	89.9
FPC [13]	discrete	w/o	72.9	82.9	91.3	90.4
GenPT [39]	discrete	w/o	74.7	83.4	91.1	-
KnowPrompt [38]	continuous	w/o	72.4	82.4	91.3	90.2
PTCAS(ours)	continuous	w/o	73.5	83.2	91.4	90.3

Table 4

Experimental results for low-resource RE on different datasets. The best results are in bold and second-best are underlined.

Model	TACRED			TACREV			ReTACRED		
	K=8	K=16	K=32	K=8	K=16	K=32	K=8	K=16	K=32
SpanBERT	8.4	17.5	17.9	5.2	5.7	18.6	14.2	29.3	43.9
LUKE	9.5	21.5	28.7	9.8	22.0	29.3	14.1	37.5	52.3
GDPNet	11.8	22.5	28.8	8.3	20.8	28.1	18.8	48.0	54.8
TYP Marker	28.9	32.0	32.4	27.6	31.2	32.0	44.8	54.1	60.0
PTR	28.1	30.7	32.1	28.7	31.4	32.4	51.5	56.2	62.1
KnowPrompt	32.0	35.4	36.5	32.1	33.1	34.7	55.3	<u>63.3</u>	65.0
FPC	<u>33.6</u>	34.7	35.8	<u>33.1</u>	34.3	35.5	57.9	60.4	<u>65.3</u>
GenPT	35.7	36.6	37.4	34.4	34.6	36.2	<u>57.1</u>	60.4	65.2
PTCAS (ours)	32.8	<u>35.7</u>	<u>37.2</u>	32.9	<u>34.4</u>	<u>35.7</u>	55.9	63.8	65.3

5.3. Implementation details

Our model was implemented using PyTorch [48] and the HuggingFace transformer library [49]. We built our PTCAS models based on a pre-trained *ROBERTA_{LARGE}* language model [5]. For the optimization procedure, ADAMW [50] was adopted as the optimizer. The soft-prompt token lengths L_s and [R] token lengths L_r were set to 5 and 7, respectively. The margin γ was set to 1.0. Hyperparameters λ and β were set to 0.1 and 0.7, respectively.

5.4. Main results

Results of Standard Setting. As shown in Table 3, knowledge-enhanced PLMs are more effective than vanilla fine-tuning, indicating that introducing additional task-specific knowledge into models is beneficial. Notably, our model achieved significant gains over all fine-tuned models, including knowledge-enhanced models that rely on knowledge for data augmentation or architecture enhancement. These observations show that it is inefficient and challenging fine-tuning methods to stimulate knowledge of PLMs for downstream tasks. Compared to existing discriminative prompt-tuning methods with a discrete answer space, our method achieved better results for most datasets, indicating that it can arrive at a better optimization state in a continuous answer space. Compared to the generative prompt-tuning method GenPT [39], which is the current state-of-the-art method, the proposed PTCAS achieved competitive performance without manual answer engineering. Additionally, although KnowPrompt [38] searches for answers in a continuous space, our model yields better results, proving its effectiveness in fully exploiting relational semantics.

Results of Low-Resource Setting. As shown in Table 4, our model achieves a significant improvement over all fine-tuning methods under few-shot settings. Compared to prompt-tuning methods, our approach can achieve competitive or superior performance in different settings. Owing to the additional parameters required in our model, the performance of PTCAS showed a larger improvement as K increased from 8 to 32. Our model outperformed previous prompt-based methods with a discrete answer search under the low-resource settings of ReTACRED, proving the effectiveness of our proposed continuous answer search.

Table 5
Ablation study on ReTACRED.

Model	K=8	K=16	K=32	FULL
PTCAS	55.9	63.8	65.3	91.4
PTCAS($L_r = 3$)	55.2	63.3	64.8	91.1
-CAS	51.6	56.4	62.4	90.9
-prototype initialization	53.3	62.7	64.2	91.2
-transH-based constraint	55.0	62.8	64.5	91.0
with TransE-based constraint	55.5	63.4	65.1	91.3

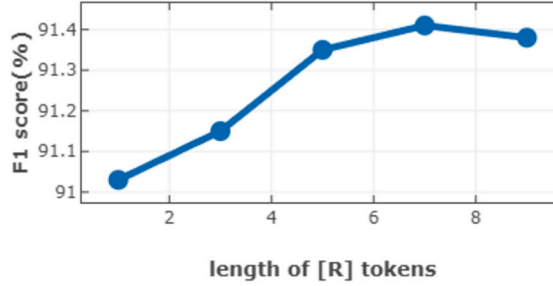


Fig. 2. Performance with different numbers of [R] tokens on ReTACRED.

6. Analysis and discussion

In this section, we discuss the influence of each module on our method and provide details of the ablation experiments. Additionally, we investigate the impact of [R] token length.

6.1. Effect of continuous answer search

To demonstrate the effectiveness of the continuous answer search in our prompt-based method, we conducted an ablation study on ReTACRED and reported the results of the different models in Table 5. Specifically, in the -CAS setting, we used a well-designed discrete answer word space and mapped it to the relation label set from PTR [12], where three words were selected elaborately for each relation label. For a fair comparison, the number of [R] tokens in PTCAS ($L_r = 3$) was set to 3. The results show that, compared to a well-designed discrete answer space search, a continuous answer search achieves better performance in both full- and low-resource settings, indicating that our continuous answer search can find optimal embeddings for relations without requiring manual design.

6.2. Effect of prototype initialization

To illustrate the effect of our prototype initialization on relation types, an ablation study was conducted, and the results are presented in the third row of Table 5. In the -prototype initialization setting, we randomly initialized relation-type embeddings instead of using prototype words. The results reveal that without prototype words for initialization, the model performs worse, especially in few-shot scenarios, which indicates that knowledge of prototype words is indispensable for optimization.

6.3. Effect of TransH-based constraint

We also conducted an ablation study on the effects of the TransH-based constraints. According to Table 5, removing the TransH-based constraint decreases the F1 score. Furthermore, the with TransE-based constraint model was obtained by replacing the TransH-based constraint with a TransE-based constraint. Evidently, the with TransE-based constraint model achieves an improvement in performance compared to the model without any structured constraint. Moreover, our PTCAS model achieved better performance than the with TransE-based constraint model, demonstrating the effectiveness of our TransH-based constraint.

6.4. Analysis of [R] token length

In this section, we analyze the effect of [R] token length. A series of experiments were conducted to determine the number of [R] tokens required to encode sufficient semantic information for relation classification. In our implementation, the lengths of the [R] tokens, denoted by L_r in template $\mathcal{T}(\cdot)$, were set to 1, 3, 5, 7, and 9. The results are shown in Fig. 2. We observed that the F1 score increased as n_r increased from 0 to 7 and then decreased. This indicates that seven [R] tokens were sufficient for relation semantic encoding, which explains why $L_r = 7$ was the default setting.

7. Conclusion

In this study, we introduced a novel prompt-tuning approach with a continuous answer search for relation extraction that enables the model to determine the optimal answer word embeddings in a continuous space through gradient descent, fully exploiting relational semantics. Furthermore, we designed and incorporated an additional TransH-based structured knowledge constraint into the optimization procedure, allowing the model to mine the structured knowledge of the entity type. A series of experiments was conducted on four widely used RE benchmarks. The experimental results show that our approach achieves competitive or superior performance without manual answer engineering compared to existing baselines under both fully supervised and low-resource scenarios. Further ablation analysis demonstrated the effectiveness of the proposed continuous answer search mechanism and TransH-based structured knowledge constraint. In summary, our work provides deeper research towards utilizing prompt tuning with continuous answer search for relation extraction, which could be a practical learning paradigm. Our future work will include exploring more efficient prompt templates and applying the proposed method to additional datasets for relational extraction.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] A. Bordes, N. Usunier, A. Garcia-Duran, J. Weston, O. Yakhnenko, Translating embeddings for modeling multi-relational data, *Adv. Neural Inf. Process. Syst.* 26 (2013).
- [2] A. Madotto, C.-S. Wu, P. Fung, Mem2seq: effectively incorporating knowledge bases into end-to-end task-oriented dialog systems, *arXiv preprint arXiv:1804.08217*, 2018.
- [3] A. Bordes, S. Chopra, J. Weston, Question answering with subgraph embeddings, *arXiv preprint arXiv:1406.3676*, 2014.
- [4] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, Bert: pre-training of deep bidirectional transformers for language understanding, *arXiv preprint arXiv:1810.04805*, 2018.
- [5] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, V. Stoyanov, Roberta: a robustly optimized bert pretraining approach, *arXiv preprint arXiv:1907.11692*, 2019.
- [6] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, G. Neubig, Pre-train, prompt, and predict: a systematic survey of prompting methods in natural language processing, *ACM Comput. Surv.* 55 (9) (2023) 1–35.
- [7] T. Brown, B. Mann, N. Ryder, M. Subbiah, J.D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., Language models are few-shot learners, *Adv. Neural Inf. Process. Syst.* 33 (2020) 1877–1901.
- [8] T. Schick, H. Schütze, Exploiting cloze questions for few shot text classification and natural language inference, *arXiv preprint arXiv:2001.07676*, 2020.
- [9] X. Liu, Y. Zheng, Z. Du, M. Ding, Y. Qian, Z. Yang, J. Tang, Gpt understands, too, *arXiv preprint arXiv:2103.10385*, 2021.
- [10] X.L. Li, P. Liang, Prefix-tuning: optimizing continuous prompts for generation, *arXiv preprint arXiv:2101.00190*, 2021.
- [11] Q. Li, Y. Wang, T. You, Y. Lu, Bioknowprompt: incorporating imprecise knowledge into prompt-tuning verbalizer with biomedical text for relation extraction, *Inf. Sci.* 617 (2022) 346–358.
- [12] X. Han, W. Zhao, N. Ding, Z. Liu, M. Sun, Ptr: prompt tuning with rules for text classification, *AI Open* 3 (2022) 182–192.
- [13] S. Yang, D. Song, Fpc: fine-tuning with prompt curriculum for relation extraction, in: *Proceedings of the 2nd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 12th International Joint Conference on Natural Language Processing, 2022*, pp. 1065–1077.
- [14] R. Mooney, Relational learning of pattern-match rules for information extraction, in: *Proceedings of the Sixteenth National Conference on Artificial Intelligence*, vol. 328, 1999, p. 334.
- [15] Z. Geng, J. Li, Y. Han, Y. Zhang, Novel target attention convolutional neural network for relation classification, *Inf. Sci.* 597 (2022) 24–37.
- [16] Q. Sun, K. Zhang, K. Huang, T. Xu, X. Li, Y. Liu, Document-level relation extraction with two-stage dynamic graph attention networks, *Knowl.-Based Syst.* 267 (2023) 110428.
- [17] M.E. Peters, M. Neumann, R.L. Logan IV, R. Schwartz, V. Joshi, S. Singh, N.A. Smith, Knowledge enhanced contextual word representations, *arXiv preprint arXiv:1909.04164*, 2019.
- [18] M. Joshi, D. Chen, Y. Liu, D.S. Weld, L. Zettlemoyer, O. Levy, Spanbert: improving pre-training by representing and predicting spans, *Trans. Assoc. Comput. Linguist.* 8 (2020) 64–77.
- [19] I. Yamada, A. Asai, H. Shindo, H. Takeda, Y. Matsumoto, Luke: deep contextualized entity representations with entity-aware self-attention, *arXiv preprint arXiv:2010.01057*, 2020.
- [20] L.B. Soares, N. FitzGerald, J. Ling, T. Kwiatkowski, Matching the blanks: distributional similarity for relation learning, *arXiv preprint arXiv:1906.03158*, 2019.
- [21] Y. Li, Z. Ma, L. Gao, Y. Wu, F. Xie, X. Ren, Enhance prototypical networks with hybrid attention and confusing loss function for few-shot relation classification, *Neurocomputing* 493 (2022) 362–372.
- [22] H. Du, Q. Zhang, K. Li, F. Zhao, G. Wang, Multi-transformer based on prototypical enhancement network for few-shot relation classification with domain adaptation, *Neurocomputing* 559 (2023) 126796.
- [23] M. Wen, T. Xia, B. Liao, Y. Tian, Few-shot relation classification using clustering-based prototype modification, *Knowl.-Based Syst.* 268 (2023) 110477.
- [24] J. Snell, K. Swersky, R. Zemel, Prototypical networks for few-shot learning, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [25] S. Yang, Y. Zhang, G. Niu, Q. Zhao, S. Pu, Entity concept-enhanced few-shot relation extraction, *arXiv preprint arXiv:2106.02401*, 2021.
- [26] Y. Liu, J. Hu, X. Wan, T.-H. Chang, A simple yet effective relation information guided approach for few-shot relation extraction, *arXiv preprint arXiv:2205.09536*, 2022.
- [27] Y. Liu, J. Hu, X. Wan, T.-H. Chang, Learn from relation information: towards prototype representation rectification for few-shot relation extraction, in: *Findings of the Association for Computational Linguistics: NAACL 2022*, 2022, pp. 1822–1831.
- [28] F. Petroni, T. Rocktäschel, P. Lewis, A. Bakhtin, Y. Wu, A.H. Miller, S. Riedel, Language models as knowledge bases?, *arXiv preprint arXiv:1909.01066*, 2019.

- [29] T. Schick, H. Schütze, Few-shot text generation with pattern-exploiting training, arXiv preprint arXiv:2012.11926, 2020.
- [30] T. Gao, A. Fisch, D. Chen, Making pre-trained language models better few-shot learners, arXiv preprint arXiv:2012.15723, 2020.
- [31] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, P.J. Liu, Exploring the limits of transfer learning with a unified text-to-text transformer, *J. Mach. Learn. Res.* 21 (1) (2020) 5485–5551.
- [32] T. Shin, Y. Razeghi, R.L. Logan IV, E. Wallace, S. Singh, Autoprompt: eliciting knowledge from language models with automatically generated prompts, arXiv preprint arXiv:2010.15980, 2020.
- [33] Z. Zhong, D. Friedman, D. Chen, Factual probing is [mask]: learning vs. learning to recall, arXiv preprint arXiv:2104.05240, 2021.
- [34] L. Cui, Y. Wu, J. Liu, S. Yang, Y. Zhang, Template-based named entity recognition using bart, arXiv preprint arXiv:2106.01760, 2021.
- [35] W. Yin, J. Hay, D. Roth, Benchmarking zero-shot text classification: datasets, evaluation and entailment approach, arXiv preprint arXiv:1909.00161, 2019.
- [36] Z. Jiang, F.F. Xu, J. Araki, G. Neubig, How can we know what language models know?, *Trans. Assoc. Comput. Linguist.* 8 (2020) 423–438.
- [37] K. Hambardzumyan, H. Khachatryan, J. May, Warp: word-level adversarial reprogramming, arXiv preprint arXiv:2101.00121, 2021.
- [38] X. Chen, N. Zhang, X. Xie, S. Deng, Y. Yao, C. Tan, F. Huang, L. Si, H. Chen, Knowprompt: knowledge-aware prompt-tuning with synergistic optimization for relation extraction, in: *Proceedings of the ACM Web Conference 2022*, 2022, pp. 2778–2788.
- [39] J. Han, S. Zhao, B. Cheng, S. Ma, W. Lu, Generative prompt tuning for relation classification, arXiv preprint arXiv:2210.12435, 2022.
- [40] Z. Wang, J. Zhang, J. Feng, Z. Chen, Knowledge graph embedding by translating on hyperplanes, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 28, 2014.
- [41] Y. Zhang, V. Zhong, D. Chen, G. Angeli, C.D. Manning, Position-aware attention and supervised data improve slot filling, in: *Conference on Empirical Methods in Natural Language Processing*, 2017.
- [42] C. Alt, A. Gabrysak, L. Hennig, Tacred revisited: a thorough evaluation of the tacred relation extraction task, arXiv preprint arXiv:2004.14855, 2020.
- [43] G. Stoica, E.A. Platanios, B. Póczos, Re-tacred: addressing shortcomings of the tacred dataset, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, 2021, pp. 13843–13850.
- [44] I. Hendrickx, S.N. Kim, Z. Kozareva, P. Nakov, D.O. Séaghdha, S. Padó, M. Pennacchiotti, L. Romano, S. Szpakowicz, Semeval-2010 task 8: multi-way classification of semantic relations between pairs of nominals, arXiv preprint arXiv:1911.10422, 2019.
- [45] F. Xue, A. Sun, H. Zhang, E.S. Chng, Gdpnet: refining latent multi-view graph for relation extraction, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, 2021, pp. 14194–14202.
- [46] W. Zhou, M. Chen, An improved baseline for sentence-level relation extraction, arXiv preprint arXiv:2102.01373, 2021.
- [47] O. Sainz, O.L. de Lacalle, G. Labaka, A. Barrena, E. Agirre, Label verbalization and entailment for effective zero-and few-shot relation extraction, arXiv preprint arXiv:2109.03659, 2021.
- [48] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al., Pytorch: an imperative style, high-performance deep learning library, *Adv. Neural Inf. Process. Syst.* 32 (2019).
- [49] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, et al., Transformers: state-of-the-art natural language processing, in: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45.
- [50] I. Loshchilov, F. Hutter, Decoupled weight decay regularization, arXiv preprint arXiv:1711.05101, 2017.